COMP1531
Software Engineering Fundamentals

Week 3

Correctness - Static Verification

In This Lecture

– – –

Why?

- The best time to improve software safety is **before** the code runs

What?

- Type Safety
- Typescript
- Examples

Unexpected Input

Sometimes we write a really nice function like this. And everyone uses it correctly. .

```
1 function manyString(repeat, str) {
2   let outString = '';
3   for (let i = 0; i < repeat; i++) {
4     outString += str;
5   }
6   return outString;
7 }
8 console.log(manyString('hello ', 5));
```

many_string_rude.js

..right?

Unexpected Input

Wrong! Users of your functions will often make mistakes using them.

```
1 function manyString(repeat, str) {
2   let outString = '';
3   for (let i = 0; i < repeat; i++) {
4     outString += str;
5   }
6   return outString;
7 }
8 console.log(manyString('hello ', 5));
```

many_string_rude.js

This program prints out nothing... the worst mistake a program makes is one that does not cause an error.

Unexpected Input

How can we protect against this?

Unexpected Input

```
1  function manyString(repeat, str) {
2    if (!(repeat instanceof 'number')) {
3      console.error('repeat argument is not a number');
4      return undefined;
5    }
6    if (!(str instanceof 'string')) {
7      console.error('str argument is not a string');
8      return undefined;
9    }
10   let outString = '';
11   for (let i = 0; i < repeat; i++) {
12     outString += str;
13   }
14   return outString;
15 }
16 console.log(manyString('hello ', 5));
```

We'll add a **type check** in during runtime to check the type being passed in.

many_string_runtime.js

Type Safety

- **Preventing mismatches** between the actual and expected type of variables, constants and functions
- **C is type-safe***, as types must be declared and the compiler will check that the types are correct
- Javascript, on its own, is **not** type-safe. Everything has a type, but that type is not known till the program is executed.

Type Safety In Javascript

– – –

- The solution we saw previously is what we would refer to as improving software safety dynamically - by "**catching**" issues **at runtime**.
- However, rather than dynamically checking for certain errors, it is always better if errors can be detected statically.
- We need a way to check for correct types statically in Javascript.

Typescript

- Typescript is a language built on top of Javascript. It's job is to check the types in your program and outputs Javascript that is then run with node.

```ts
function sum(a: number, b: number) {
  return a + b;
}
console.log(sum(1, 2));

// const a: Number = 1;
// const b: String = "q";
// const c: Boolean = true;
```

mycode.ts

But how do I run this code?

Typescript

Typescript is another dependency we need to install:

```
npm install --save-dev tsx typescript@5.9.3
```

Note: we are not using the latest version of typescript, hence we specify the version we will use with the @ symbol

Typescript

Running node With Typescript

Once this is installed, we can run our typescript code (e.g. `mycode.ts`) with the following command:

```
node_modules/.bin/tsx mycode.ts
```

Typescript

Type Checking With Typescript

Whilst tsx does some type checking, it also runs the code. It's useful to have a way to "type check without running" that also checks a bit more strictly.

```
node_modules/.bin/tsc mycode.ts
```

Typescript

In reality you would normally add both of these commands to your `package.json`

```
"scripts": {
    "tsc": "tsc",
    "tsx": "tsx"
}
```

Typescript

Now let's try and use **`tsc`** on a program that has type errors in it! Like our original program. But let's write it in typescript.

```
1 function manyString(repeat: number, str: string) {
2   let outString = '';
3   for (let i = 0; i < repeat; i++) {
4     outString += str;
5   }
6   return outString;
7 }
8 console.log(manyString('hello ', 5));
```

`npm run tsc mycode_broken.ts`

Let's see what it outputs!

How To Typescript

Types are added to programs typically by putting the type name after a colon. We've seen that in our first example.

```
1 function sum(a: number, b: number) {
2   return a + b;
3 }
4 console.log(sum(1, 2));
5
6 // const a: Number = 1;
7 // const b: String = "q";
8 // const c: Boolean = true;
```

mycode.ts

How To Typescript

Typescript doesn't require you to put types on everything. It will infer types that it can, but sometimes it's unable to.

Typescript doesn't know what name is, you need to give it a type!

```
1 function hello(name: string) {
2   console.log(name);
3 }
```

Typescript doesn't need to be told name's type. It will figure it out:

```
1 const myName = 'Yuchao'
```

Examples Of Typing

Basics & Function

The most basic 3 types in Typescript are string, number, and boolean.

Functions typically require **all parameters** to be explicitly typed.

```
1 function hello(name: string) {
2   return `Hello ${name}!`;
3 }
```

example_functions.ts

Examples Of Typing

Unions

```
1 function printIfReady(ready: boolean | number) {
2   if (ready === true || (!ready && ready !== 0)) {
3     console.log('Ready!');
4   }
5 }
6 printIfReady(1);
7 printIfReady(0);
8 printIfReady(true);
9 printIfReady(false);
```

example_unions.ts

Examples Of Typing

Lists

```
1 function create10List(item: number) {
2   const arr: number[] = [];
3   for (let i = 0; i < 10; i++) {
4     arr.push(item);
5   }
6   return arr;
7 }
8
9 console.log(create10List(1));
```

example_lists.ts

Examples Of Typing

Aliases

```
1  type ListItem = string | number;
2
3  function create10List2(item: ListItem) {
4    const arr: ListItem[] = [];
5    for (let i = 0; i < 10; i++) {
6      arr.push(item);
7    }
8    return arr;
9  }
10
11 console.log(create10List2(17));
```

example_aliases.ts

Examples Of Typing

Optionals

```
// Note:
//   end?: number
// = end: number | undefined

function substring(str: string, start: number, end?: number) {
  let newString = '';
  const modifiedEnd = end ?? str.length;
  //nullish coalescing operator (??),
  //which only falls back if end is null or undefined
  for (let i = start; i < modifiedEnd; i++) {
    newString += str[i];
  }
  return newString;
}

console.log(substring('elephant', 0, 2));
console.log(substring('elephant', 2));
```

example_optionals.ts

Examples Of Typing

Objects

```ts
type Person = {
  name: string;
  age?: number;
  height?: number;
}

const person: Person = {
  name: "nick",
  height: 188,
};
person.age = 5;
```

example_objects.ts

Examples Of Typing

Literals

```
1 type visibility = 'Private' | 'Public';
2
3 function createChannel(name: string, visibility: visibility) {
4   // Do things
5 }
```

example_literals.ts

Examples Of Typing

Any

any is a type that kind of makes typescript pointless. It's great for a stub and a "I will come back to this later"

```ts
// @ts-nocheck

function hello(name: any): any {
  return `Hello ${name}!`;
}

function substring(str: any, start: any, end: any) {
  return null;
}

type Person = any;
const person: Person = {
  name: 'Hayden',
};
```

example_any.ts

Examples Of Typing

– – –

A much more thorough array of types and their explanations can be found on the typescript website.

Type Safety

A summary of languages and their type safety:

TypeScript Error Categories (at compile time)

- **Type errors** – Assigning a string to a number
- **Property errors** – Missing a required object property
- **Argument errors** – Passing the wrong type to a function
- **Return type errors** – Returning a string when a number is expected
- **Null/Undefined errors** – Assigning null to a non-nullable variable
- **Union type errors** – Using a method without narrowing a union type
- **Interface/type compatibility errors** – A class not fully implementing an interface
- **Enum errors** – Assigning a value not defined in the enum
- **Generic type errors** – Providing the wrong type argument to a generic
- **Module/import errors** – Importing from a module that doesn't exist
- **Access modifier errors** – Accessing a private property outside its class
- **Duplicate identifier errors** – Declaring the same variable more than once
- **Syntax errors** – Missing brackets or incorrect TypeScript syntax

Migrating To Typescript

Included below are some of the steps to get typescript working:

1. Run **`npm install --save-dev tsx typescript@5.9.3`**. This allows Typescript to work with your vitest files.
2. Add files `tsconfig.json` with course-provided info in them.
3. Update `package.json` to include a script tsc that just runs **`tsc --noImplicitAny --noEmit`** and `tsx` that just runs **`tsx`**.

Feedback

Or go to the form here

COMP1531
Software Engineering Fundamentals

Week 3

Correctness - Linting

In this Lecture

– – –

Why?

- You can't manually fix your style forever - we need automated approaches

What?

- Linting
- eslint

Coding Style

"**Programs must be written for people to read, and only incidentally for machines to execute**"

- Abelson & Sussman, "Structure and Interpretation of Computer Programs

Coding Style

– – –

- Why do we care about style?
  - It's **easier to follow** the flow of code with consistent whitespace.
  - It's **easier to visually glance** at code with similar patterns.
  - It can be **easier to detect bugs**.
    - E.G. If you force constants to be uppercase named, it's easy to spot a mutated uppercase variable.

Coding Style

Bad

```
1 function loophello(b, c){
2 let d = '';for (let i = 0; i < b; i++)
3         d += c;
4         return d;
5 }
6 console.log(loophello(5, 'hello '));
```

style_bad.js

Good

```
1 function manyString(repeat, str) {
2   let outString = '';
3   for (let i = 0; i < repeat; i++) {
4     outString += str;
5   }
6   return outString;
7 }
8 console.log(manyString(5, 'hello '));
```

style_good.js

Well Who Decides On Style??

— — —

Typically when a group picks a style, they:

1. Choose a style guide **set out by a large organisation** (E.G. Google, Facebook, Microsoft, etc).
2. (Optionally) modify specific style rules to satisfy any clear **subjective opinions** (e.g. if most of an organisation strongly believes in spaces over tabs).

In COMP1531, the staff use standard style guides and provides them to students.

Linters: Enforcing Style

In early computing courses you're given a style guide and told to **manually** make sure your code complies to style.

In proper software engineering projects we tend to "lint" code by using software that statically analyses your code and **automatically makes adjustments**. We call programs that lint code "linters". You can loosely consider them another form of static verification.

What Does A Linter Do?

– – –

Recap: **Static verification** is the processing of analysing as much of your program as you can before running it.

E.g. C compilation contains elements of static analysis (e.g. *type checking* , *unused variables*, etc)

A linter does static analysis to identify:

- **Style issues** (whitespace, indentation)
- **Semantic issues** (bad logic, potential bugs)

What Does A Linter Do?

Linting does **not** help with poorly named variables. That's still up to humans.

Because JS is interpreted (no compile step) linting helps bridge the gap of some things missed out by compilers.

`eslint`

- A popular external tool for **statically analysing javascript code**
- Can detect errors, warn of potential errors and check against conventions, or other problematic areas
- Can also automatically fix issues with your style
- Can be **configured** to be as strict or lenient as desired.

`eslint`

Installation

- `npm install --save-dev eslint`

Once again, we use `--save-dev` since the library is only used in development and not for production code.

- Install a few more plugins necessary for eslint to work with typescript and vitest:

```
$ npm install --save-dev @eslint/js typescript-eslint @typescript-eslint/eslint-plugin
@stylistic/eslint-plugin @vitest/eslint-plugin globals
```

eslint

Configuration

`eslint` determines what is and isn't OK by looking at a `.eslint.config.js`.

There are tools that will help you create your own! But for COMP1531 we are providing one.

eslint

Using eslint:

- You can run eslint on a file by the following command:
  - **`node_modules/.bin/eslint style_bad.js`**
- If nothing prints on the terminal, your file is all good!
- Semantic issues need to be fixed manually (e.g. undefined variable).
- Style issues can be fixed manually...

Eslint --fix

Automatically Fixing Style Issues

If we add a `--fix` flag to our eslint command, `eslint` will be able to fix style issues for us.

```
node_modules/.bin/eslint --fix style_bad.js
```

This **overwrites** the file.

eslint

Ignoring One-Off Issues

If you want to **disable `eslint` rules** for one-off instances, you can disable certain rules with comments as per the [eslint disabling rules guide](#).

```
1 const a=5;
2 // eslint-disable-next-line
3 const b=5;
```

In COMP1531 we only allow the use of a few `eslint-disable-next-line` directives per project, but they NEED to be checked with your tutor first.

Using It In Your Project

You will want to add the following to your `package.json` scripts.

This allows us to run **`npm run lint`** to check our code, and **`npm run lint-fix`** to check and fix (where it can).

The reason you can't lint-fix all the time is because it is slower.

Linting In COMP1531

`eslint` will be part of your **labs** from week 4. We will give you both the configuration file and add a lint command to your `package.json` scripts as well as your `eslint.config.mjs`

`eslint` will be part of the **project** from iteration 2. We will give you the configuration file but expect you to add a lint command to your `package.json` scripts and to your `eslint.config.mjs`

Finishing The Eslint Setup

All of the examples in this lecture were relatively simple.

We sadly need to do slightly more than npm install **`--save-dev`** `eslint` to get things working with Vitest and typescript.

Don't Stress!

– – –

All of the environmental setup or changes you've seen in this lecture will either be **done for you** or will be given to you with clear unambiguous instructions.

We don't expect you to all be experts in tweaking these environments.

Reminders

– – –

- Iteration 1 Released
  - Iteration 1 should use javascript and not typescript
  - Linting is optional

Feedback

Or go to the form here